```
================================================================================
         LEARNING HUB — ONLINE LEARNING PLATFORM
         30-Day Development Plan | 4 Weeks | 6 Modules
         Tech Stack: Python Django + React.js + SQLite/MySQL
================================================================================

--------------------------------------------------------------------------------
OVERVIEW
--------------------------------------------------------------------------------

  Total Days   : 30
  Total Weeks  : 4
  Modules      : 6
  Backend      : Python Django + Django REST Framework
  Frontend     : React.js + Axios + React Router
  Database     : SQLite (dev) → MySQL (production)
  Auth         : JWT (djangorestframework-simplejwt)
  Deployment   : Render (backend) + Vercel (frontend)

  MODULE LIST:
  Module 1 — User Management
  Module 2 — Course Management
  Module 3 — Learning Delivery
  Module 4 — Assessment (Quizzes & Assignments)
  Module 5 — Student Dashboard
  Module 6 — Feedback & Reviews

================================================================================
WEEK 1 — SETUP & USER AUTHENTICATION (Days 1-7)
================================================================================

--------------------------------------------------------------------------------
DAY 1 — Project Environment Setup
--------------------------------------------------------------------------------
Goal      : Get Django running and admin panel accessible

Tasks:
  [x] Install Python (3.11+) and VS Code extensions
        - Python by Microsoft
        - Pylance by Microsoft
        - Django by Baptiste Darthenay
  [x] Create project folder: D:\Online_lear_plt
  [x] Create subfolders: backend/ and frontend/
  [x] Create and activate virtual environment
        Command: python -m venv venv
        Activate (Windows): venv\Scripts\activate
```

```
  [x] Install all required packages:
        pip install django djangorestframework
                    djangorestframework-simplejwt
                    django-cors-headers Pillow python-dotenv
  [x] Save packages: pip freeze > requirements.txt
  [x] Create Django project: django-admin startproject learning_hub .
  [x] Run initial migrations: python manage.py migrate
  [x] Create superuser: python manage.py createsuperuser
  [x] Start server: python manage.py runserver

End of Day Goal:
  → Django admin panel accessible at http://127.0.0.1:8000/admin

--------------------------------------------------------------------------------
DAY 2 — Django Apps & Settings Configuration
--------------------------------------------------------------------------------
Goal      : Create all 4 apps and configure settings.py fully

Tasks:
  [x] Create 4 Django apps:
        python manage.py startapp users
        python manage.py startapp courses
        python manage.py startapp enrollments
        python manage.py startapp assessments
  [x] Add apps to INSTALLED_APPS in settings.py:
        'rest_framework', 'corsheaders',
        'users', 'courses', 'enrollments', 'assessments'
  [x] Add CorsMiddleware at top of MIDDLEWARE
  [x] Configure CORS_ALLOWED_ORIGINS = ["http://localhost:3000"]
  [x] Set REST_FRAMEWORK default auth to JWTAuthentication
  [x] Set AUTH_USER_MODEL = 'users.CustomUser'
  [x] Set MEDIA_URL and MEDIA_ROOT for file uploads
  [x] Create .env file with SECRET_KEY and DEBUG=True
  [x] Load .env in settings.py using python-dotenv

End of Day Goal:
  → All apps created, settings fully configured, server still running

--------------------------------------------------------------------------------
DAY 3 — Custom User Model (MODULE 1 START)
--------------------------------------------------------------------------------
Module    : Module 1 — User Management
Goal      : Build CustomUser model with 3 roles

Tasks:
  [x] Write CustomUser model in users/models.py
        - Extend AbstractUser
```

```
        - Add role field with choices: admin, instructor, student
        - Add bio (TextField) and profile_picture (ImageField)
  [x] Write UserSerializer in users/serializers.py
  [x] Register CustomUser in users/admin.py using CustomUserAdmin
        - Add list_display: username, email, role, is_staff
  [x] Run: python manage.py makemigrations users
  [x] Run: python manage.py migrate
  [x] Verify role column appears in admin panel

End of Day Goal:
  → CustomUser model visible in admin with Role column

---------------------------------------------------------------------------
DAY 4 — Register & Login API
---------------------------------------------------------------------------
Module    : Module 1 — User Management
Goal      : Build working register and login endpoints with JWT

Tasks:
  [x] Write RegisterSerializer in users/serializers.py
        - Fields: username, email, password, role
        - Use create_user() to hash password
  [x] Write RegisterView in users/views.py
        - POST /api/users/register/
        - permission_classes = [AllowAny]
  [x] Write LoginView in users/views.py
        - POST /api/users/login/
        - Returns JWT access + refresh tokens + role
  [x] Write ProfileView in users/views.py
        - GET /api/users/profile/
        - permission_classes = [IsAuthenticated]
  [x] Create users/urls.py with routes:
        - register/, login/, profile/
  [x] Include users.urls in learning_hub/urls.py
  [x] Test register via browser at:
        http://127.0.0.1:8000/api/users/register/

End of Day Goal:
  → Register and login return real JWT access tokens

---------------------------------------------------------------------------
DAY 5 — Role-Based Permissions
---------------------------------------------------------------------------
Module    : Module 1 — User Management
Goal      : Protect routes based on user role

Tasks:
```

```
  [ ] Create users/permissions.py with custom permission classes:
        - IsAdmin: user.role == 'admin'
        - IsInstructor: user.role == 'instructor'
        - IsStudent: user.role == 'student'
  [ ] Add profile update view (PATCH /api/users/profile/update/)
  [ ] Apply permission classes to views
  [ ] Test: access protected route without token → 401 Unauthorized
  [ ] Test: login as student → cannot access instructor route

End of Day Goal:
  → Role-based access enforced on all protected API routes

--------------------------------------------------------------------------------
DAY 6 — React Frontend Setup + Auth UI
--------------------------------------------------------------------------------
Module    : Module 1 — User Management
Goal      : Build Login and Register screens in React

Tasks:
  [ ] Install Node.js from nodejs.org
  [ ] Run: npx create-react-app . inside frontend/
  [ ] Install packages: npm install axios react-router-dom
  [ ] Set up React Router in App.js with routes:
        /, /login, /register, /dashboard
  [ ] Create src/services/api.js with axios base URL:
        http://localhost:8000
  [ ] Build LoginPage component
        - Fields: username, password
        - POST to /api/users/login/
  [ ] Build RegisterPage component
        - Fields: username, email, password, role dropdown
        - POST to /api/users/register/
  [ ] Start React: npm start
  [ ] Verify both pages open at http://localhost:3000

End of Day Goal:
  → Login and Register pages visible in browser

--------------------------------------------------------------------------------
DAY 7 — Connect Frontend ↔ Backend + Auth State
--------------------------------------------------------------------------------
Module    : Module 1 — User Management
Goal      : Wire React to Django API, store JWT, protect routes

Tasks:
  [ ] Connect LoginPage to POST /api/users/login/ via axios
  [ ] Save JWT access token to localStorage on login
```

```
  [ ] Create AuthContext using React Context API
        - Store: token, user, role
        - Functions: login(), logout()
  [ ] Create PrivateRoute component
        - Redirects to /login if no token found
  [ ] Protect /dashboard route with PrivateRoute
  [ ] Test full flow:
        Register → Login → Token saved → Redirect to dashboard
  [ ] Commit all Week 1 work to GitHub

End of Day Goal:
  → Full authentication flow working end-to-end ✓ MODULE 1 COMPLETE

================================================================================
WEEK 2 — COURSES & LEARNING DELIVERY (Days 8–16)
================================================================================

--------------------------------------------------------------------------------
DAY 8 — Course Model & Database
--------------------------------------------------------------------------------
Module    : Module 2 — Course Management
Goal      : Build Course, Lesson, and Material database models

Tasks:
  [ ] Write Course model in courses/models.py:
        - title, description, instructor (FK → User)
        - category, thumbnail (ImageField), created_at
  [ ] Write Lesson model:
        - title, course (FK), video_file, order_number, duration
  [ ] Write CourseMaterial model:
        - lesson (FK), file, material_type (pdf/doc/video)
  [ ] Run: python manage.py makemigrations courses
  [ ] Run: python manage.py migrate
  [ ] Register all models in courses/admin.py
  [ ] Add sample course via admin panel

End of Day Goal:
  → Course tables in database, visible and manageable from admin

--------------------------------------------------------------------------------
DAY 9 — Course CRUD API
--------------------------------------------------------------------------------
Module    : Module 2 — Course Management
Goal      : Build full Create, Read, Update, Delete API for courses

Tasks:
  [ ] Write CourseSerializer and LessonSerializer
```

```
  [ ] Write CourseListCreateView:
        - GET: all courses (public)
        - POST: instructors only
  [ ] Write CourseDetailView:
        - GET: course details + lesson list
        - PUT/DELETE: instructor who owns course only
  [ ] Write LessonCreateView (instructor only)
  [ ] Set up courses/urls.py and include in root urls.py
  [ ] Add file upload support for video_file and thumbnail
  [ ] Test: create course via API, fetch via GET

End of Day Goal:
  → Full course CRUD API working and tested

--------------------------------------------------------------------------
DAY 10 — Enrollment Model & API
--------------------------------------------------------------------------
Module    : Module 3 — Learning Delivery
Goal      : Students can enroll in courses and track progress

Tasks:
  [ ] Write Enrollment model in enrollments/models.py:
        - student (FK User), course (FK Course)
        - enrolled_at, is_completed
  [ ] Write Progress model:
        - enrollment (FK), lesson (FK)
        - is_completed (bool), completed_at
  [ ] Write EnrollView (POST /api/enrollments/enroll/)
        - Students only, prevent duplicate enrollment
  [ ] Write MyCoursesView (GET enrolled courses for logged-in student)
  [ ] Write MarkLessonCompleteView (PATCH)
        - Marks lesson done + calculates progress %
  [ ] Run migrations and test enrollment via API

End of Day Goal:
  → Students can enroll and lesson progress is tracked

--------------------------------------------------------------------------
DAY 11 — Course Listing Page (React)
--------------------------------------------------------------------------
Module    : Module 2 — Course Management (Frontend)
Goal      : Students can browse and enroll in courses

Tasks:
  [ ] Build CoursesPage:
        - Fetches GET /api/courses/
        - Renders grid of CourseCard components
```

```
  [ ] Build CourseCard component:
        - Shows thumbnail, title, instructor name, enroll button
  [ ] Add search bar to filter courses by title
  [ ] Build CourseDetailPage:
        - Shows description, instructor, lesson list
  [ ] Wire "Enroll" button to POST /api/enrollments/enroll/
  [ ] Show "Already enrolled" if student is already enrolled

End of Day Goal:
  → Students can browse all courses and click to enroll

--------------------------------------------------------------------------------
DAY 12 — Video Player & Lesson Viewer
--------------------------------------------------------------------------------
Module    : Module 3 — Learning Delivery (Frontend)
Goal      : Students can watch video lectures inside the app

Tasks:
  [ ] Build LessonPage layout:
        - Left sidebar: lesson list with checkmarks
        - Right panel: HTML5 video player
  [ ] Play uploaded video files from /media/ folder
  [ ] Add "Mark as Complete" button
        - Calls PATCH /api/enrollments/complete-lesson/
  [ ] Show progress bar at top (% of lessons completed)
  [ ] List downloadable materials below the video
  [ ] Auto-navigate to next lesson after completion

End of Day Goal:
  → Students can watch videos and track lesson progress

--------------------------------------------------------------------------------
DAY 13 — Instructor Dashboard
--------------------------------------------------------------------------------
Module    : Module 2 — Course Management (Frontend)
Goal      : Instructors can create and manage their courses

Tasks:
  [ ] Build InstructorDashboard page:
        - Shows all courses created by this instructor
        - Displays enrolled student count per course
  [ ] Build CreateCoursePage form:
        - Fields: title, description, category, thumbnail upload
        - POST to /api/courses/
  [ ] Build AddLessonPage form:
        - Fields: title, video upload, order number
  [ ] Add Edit and Delete buttons on course cards
```

```
        - Instructor can only edit/delete their own courses
  [ ] Show success/error messages after actions

End of Day Goal:
  → Instructors can fully create, edit and delete their courses

--------------------------------------------------------------------------------
DAY 14 — Student Dashboard
--------------------------------------------------------------------------------
Module    : Module 5 — Student Dashboard
Goal      : Students have a personal learning overview page

Tasks:
  [ ] Build StudentDashboard page:
        - Fetch enrolled courses from /api/enrollments/my-courses/
        - Show each course with a progress bar
  [ ] Add "Continue Learning" button:
        - Links to last incomplete lesson
  [ ] Show completed courses with a green "Completed" badge
  [ ] Add profile section:
        - Student name, role, profile picture
        - Edit profile button
  [ ] Show total courses enrolled and completed as stats

End of Day Goal:
  → Student dashboard fully functional with progress display ✓ MODULE 5 COMPLETE

--------------------------------------------------------------------------------
DAYS 15–16 — Buffer & Review Days
--------------------------------------------------------------------------------
Goal      : Fix bugs, polish Week 2 work, commit everything

Tasks:
  [ ] Re-test all auth flows (register, login, token expiry)
  [ ] Re-test enrollment and progress tracking
  [ ] Fix any broken API calls between React and Django
  [ ] Make all pages responsive on mobile (check with dev tools)
  [ ] Clean up code: remove console.log, add comments
  [ ] Make sure video playback works correctly
  [ ] Commit all Week 1 + Week 2 work to GitHub
        - Use clear commit messages

End of Day Goal:
  → Weeks 1–2 stable, committed to GitHub ✓ MODULES 2 & 3 COMPLETE

================================================================================
WEEK 3 — ASSESSMENTS & FEEDBACK (Days 17–23)
```

```
================================================================================

--------------------------------------------------------------------------------
DAY 17 — Quiz Model & Database
--------------------------------------------------------------------------------
Module    : Module 4 — Assessment
Goal      : Build Quiz, Question, Option database models

Tasks:
  [ ] Write Quiz model in assessments/models.py:
        - title, course (FK), total_marks, duration_minutes
  [ ] Write Question model:
        - quiz (FK), question_text, marks
  [ ] Write Option model:
        - question (FK), option_text, is_correct (boolean)
  [ ] Write QuizSubmission model:
        - student (FK), quiz (FK), score, submitted_at
  [ ] Run: python manage.py makemigrations assessments
  [ ] Run: python manage.py migrate
  [ ] Register all models in assessments/admin.py
  [ ] Add a sample quiz with questions via admin panel

End of Day Goal:
  → Quiz tables ready, sample quiz visible in admin panel

--------------------------------------------------------------------------------
DAY 18 — Quiz Submission & Auto-Scoring
--------------------------------------------------------------------------------
Module    : Module 4 — Assessment
Goal      : Students submit quiz, score is automatically calculated

Tasks:
  [ ] Write SubmitQuizView:
        - POST /api/assessments/quiz/submit/
        - Accepts: {quiz_id, answers: [{question_id, option_id}]}
        - Checks is_correct for each answer
        - Calculates and saves total score
  [ ] Write QuizResultView:
        - Returns score and correct answers after submission
  [ ] Write StudentGradesView:
        - Lists all quiz scores for the logged-in student
  [ ] Prevent resubmission (check if submission exists)
  [ ] Test: submit quiz via API, verify score is correct

End of Day Goal:
  → Auto-scoring works correctly, grades saved to database

```

```
--------------------------------------------------------------------------
DAY 19 — Assignment Submission System
--------------------------------------------------------------------------
Module    : Module 4 — Assessment
Goal      : Students upload files, instructors grade them

Tasks:
  [ ] Write Assignment model:
        - title, course (FK), due_date, description
  [ ] Write AssignmentSubmission model:
        - student (FK), assignment (FK)
        - submitted_file (FileField), grade, feedback
  [ ] Write SubmitAssignmentView:
        - POST: accepts file upload from student
  [ ] Write GradeAssignmentView (instructor only):
        - PATCH: instructor sets grade (0-100) + written feedback
  [ ] Write MySubmissionsView:
        - GET: student sees their submissions + grades
  [ ] Run migrations and test file upload via API

End of Day Goal:
  → Students can upload assignments, instructors can grade them

--------------------------------------------------------------------------
DAY 20 — Quiz UI in React
--------------------------------------------------------------------------
Module    : Module 4 — Assessment (Frontend)
Goal      : Students can take quizzes in the browser

Tasks:
  [ ] Build QuizPage:
        - Fetches questions from /api/assessments/quiz/{id}/
        - Shows one question at a time with radio button options
  [ ] Track selected answers in React useState
  [ ] Add countdown timer using useEffect
        - Auto-submits when timer reaches 0
  [ ] On submit: POST answers to /api/assessments/quiz/submit/
  [ ] Build QuizResultPage:
        - Shows final score (e.g. 8/10)
        - Highlights correct answers in green, wrong in red

End of Day Goal:
  → Students can take timed quizzes and see their results

--------------------------------------------------------------------------
DAY 21 — Feedback & Reviews Module
--------------------------------------------------------------------------
```

```
Module    : Module 6 — Feedback & Reviews
Goal      : Course ratings, reviews and instructor announcements

Tasks:
  [ ] Write Review model:
        - student (FK), course (FK), rating (1-5)
        - comment (TextField), created_at
  [ ] Write Announcement model:
        - instructor (FK), course (FK)
        - title, body, created_at
  [ ] Write AddReviewView (students only, one per course)
  [ ] Write CourseReviewsView (GET all reviews for a course)
  [ ] Write PostAnnouncementView (instructors only)
  [ ] Write CourseAnnouncementsView (GET announcements)
  [ ] Calculate average star rating per course
  [ ] Display average rating on CourseCard in React

End of Day Goal:
  → Reviews, ratings and announcements working ✓ MODULE 6 COMPLETE

--------------------------------------------------------------------------------
DAYS 22-23 — Assignment UI + Grades Page
--------------------------------------------------------------------------------
Module    : Module 4 — Assessment (Frontend)
Goal      : Full assignment and grading flow in the browser

Tasks:
  [ ] Build AssignmentPage:
        - Shows assignment title, description, due date
        - File upload form (PDF/DOC)
        - Submit button → POST to /api/assessments/assignment/submit/
  [ ] Build GradesPage:
        - Table of all quiz scores and assignment grades
        - Shows: subject, score, date, instructor feedback
  [ ] Build InstructorGradingPage:
        - Lists all student submissions for an assignment
        - Grade input (0-100) + feedback text box per student
        - Save button → PATCH to /api/assessments/grade/
  [ ] Test full cycle:
        Student submits → Instructor grades → Student sees grade
  [ ] Commit all Week 3 work to GitHub

End of Day Goal:
  → Full assessment cycle working end-to-end ✓ MODULE 4 COMPLETE

================================================================================
WEEK 4 — TESTING, POLISH & DEPLOYMENT (Days 24-30)
```

```
================================================================================

--------------------------------------------------------------------------------
DAY 24 — Full Application Testing
--------------------------------------------------------------------------------
Goal      : Test every user flow from start to finish

Tasks:
  [ ] Test Student flow:
        Register → Login → Browse courses → Enroll
        → Watch video → Mark complete → Take quiz → View grade
  [ ] Test Instructor flow:
        Login → Create course → Add lessons → Post assignment
        → View submissions → Grade student work
  [ ] Test Admin flow:
        Login → View all users → Manage courses → View enrollments
  [ ] Test edge cases:
        - Wrong password → error message shown
        - Empty form submit → validation errors shown
        - Duplicate enrollment → blocked with message
        - Quiz resubmission → blocked
        - Assignment past due date → show warning
  [ ] Write down all bugs found in a bug list

End of Day Goal:
  → Complete bug list documented, ready to fix

--------------------------------------------------------------------------------
DAYS 25-26 — Bug Fixing & UI Polish
--------------------------------------------------------------------------------
Goal      : Fix all bugs and improve the look and feel

Tasks:
  [ ] Fix all bugs found on Day 24
  [ ] Add loading spinner on all API calls:
        - Use React useState(loading) pattern
        - Show spinner while waiting for response
  [ ] Add proper error messages throughout:
        - "Invalid credentials" on failed login
        - "Already enrolled in this course"
        - "Quiz already submitted"
  [ ] Make all pages mobile responsive:
        - Check with browser developer tools (F12)
        - Fix any broken layouts at small screen sizes
  [ ] Build a Navbar with role-aware links:
        - Student: Home, My Courses, Grades
        - Instructor: Dashboard, Create Course
```

```
        - Admin: Users, All Courses
  [ ] Add logout button that clears token and redirects to login

End of Day Goal:
  → App is polished, responsive, no major bugs remaining

--------------------------------------------------------------------------------
DAY 27 — Prepare for Deployment
--------------------------------------------------------------------------------
Goal      : Configure the app for production environment

Tasks:
  [ ] Set DEBUG=False in .env
  [ ] Add ALLOWED_HOSTS in settings.py:
        ALLOWED_HOSTS = ['localhost', '127.0.0.1', '.render.com']
  [ ] Run: python manage.py collectstatic
  [ ] Install gunicorn: pip install gunicorn
  [ ] Update requirements.txt: pip freeze > requirements.txt
  [ ] Create .gitignore file in root:
        venv/, .env, __pycache__/, *.pyc
        node_modules/, build/, .DS_Store
  [ ] Final push of all code to GitHub
  [ ] Make sure GitHub repo has both backend/ and frontend/ folders

End of Day Goal:
  → All code pushed to GitHub, ready for live deployment

--------------------------------------------------------------------------------
DAY 28 — Deploy Backend on Render
--------------------------------------------------------------------------------
Goal      : Put Django backend live on the internet

Tasks:
  [ ] Sign up at render.com
  [ ] New Web Service → Connect your GitHub repository
  [ ] Set Build Command:
        pip install -r requirements.txt
  [ ] Set Start Command:
        gunicorn learning_hub.wsgi:application
  [ ] Add Environment Variables on Render dashboard:
        SECRET_KEY = your-secret-key
        DEBUG = False
        ALLOWED_HOSTS = .render.com
  [ ] Wait for deployment to complete
  [ ] Test live backend:
        https://your-app.onrender.com/admin
        https://your-app.onrender.com/api/users/register/
```

```
  [ ] Create superuser on Render:
        python manage.py createsuperuser (via Render shell)

End of Day Goal:
  → Django backend is live and accessible at a real URL

------------------------------------------------------------------------
DAY 29 — Deploy Frontend on Vercel
------------------------------------------------------------------------
Goal      : Put React frontend live on the internet

Tasks:
  [ ] Update src/services/api.js:
        Change base URL from http://localhost:8000
        to your Render backend URL
  [ ] Build React app: npm run build
  [ ] Sign up at vercel.com
  [ ] Import Project → Connect frontend GitHub folder
  [ ] Vercel auto-detects React → Click Deploy
  [ ] Wait for deployment (usually under 2 minutes)
  [ ] Test the full live application:
        - Register a new account
        - Login and check role redirect
        - Enroll in a course
        - Take a quiz
        - All done on live URLs!

End of Day Goal:
  → Full app live — frontend on Vercel + backend on Render

------------------------------------------------------------------------
DAY 30 — Final Review & Documentation
------------------------------------------------------------------------
Goal      : Write documentation and prepare final deliverable

Tasks:
  [ ] Write README.md file with:
        - Project title and description
        - Tech stack list
        - How to run locally (step-by-step)
        - Live demo link (Vercel URL)
        - Admin login details (for reviewer)
  [ ] Do a complete walkthrough of the live app:
        - Login as Student → test all student features
        - Login as Instructor → test course creation
        - Login as Admin → check admin panel
  [ ] Take screenshots of all key screens:
```

```
        - Login page, Student dashboard, Course page
        - Quiz page, Instructor dashboard, Admin panel
  [ ] Prepare project report / synopsis update
  [ ] Final GitHub commit with message:
        "Final submission - Learning Hub v1.0"

End of Day Goal:
  → Project complete and ready to submit ✓ ALL 6 MODULES COMPLETE

================================================================================
MODULE COMPLETION SUMMARY
================================================================================

  Module 1 — User Management       → Completed: Day 7
  Module 2 — Course Management     → Completed: Day 15
  Module 3 — Learning Delivery     → Completed: Day 16
  Module 4 — Assessment            → Completed: Day 23
  Module 5 — Student Dashboard     → Completed: Day 14
  Module 6 — Feedback & Reviews    → Completed: Day 21

================================================================================
API ENDPOINTS SUMMARY
================================================================================

  USER ENDPOINTS:
  POST   /api/users/register/          → Register new user
  POST   /api/users/login/             → Login + get JWT token
  GET    /api/users/profile/           → Get logged-in user profile
  PATCH  /api/users/profile/update/    → Update profile

  COURSE ENDPOINTS:
  GET    /api/courses/                 → List all courses
  POST   /api/courses/                 → Create course (instructor)
  GET    /api/courses/{id}/            → Course detail + lessons
  PUT    /api/courses/{id}/            → Update course (owner only)
  DELETE /api/courses/{id}/            → Delete course (owner only)

  ENROLLMENT ENDPOINTS:
  POST   /api/enrollments/enroll/      → Enroll in a course
  GET    /api/enrollments/my-courses/  → Student's enrolled courses
  PATCH  /api/enrollments/complete-lesson/ → Mark lesson complete

  ASSESSMENT ENDPOINTS:
  GET    /api/assessments/quiz/{id}/   → Get quiz questions
  POST   /api/assessments/quiz/submit/ → Submit quiz answers
  GET    /api/assessments/grades/      → Student's all grades
  POST   /api/assessments/assignment/submit/ → Upload assignment
```

```
  PATCH  /api/assessments/grade/        → Grade assignment (instructor)

  FEEDBACK ENDPOINTS:
  POST   /api/feedback/review/          → Add course review
  GET    /api/feedback/reviews/{id}/    → Get course reviews
  POST   /api/feedback/announcement/    → Post announcement (instructor)

================================================================================
DAILY RULES TO FOLLOW
================================================================================

  1. Always activate venv before any Django command:
       venv\Scripts\activate

  2. Always check for (venv) prefix in terminal before running Python

  3. Commit to GitHub at least once every day

  4. Test each API endpoint before building the frontend for it

  5. Keep two terminal tabs open at all times:
       Tab 1: Django server (python manage.py runserver)
       Tab 2: React server (npm start)

  6. Never commit the .env file to GitHub

  7. If you get an error — read the last line first, it tells you what broke

================================================================================
                     END OF 30-DAY PLAN
          Learning Hub — Online Learning Platform
================================================================================
```